

10 – PACKAGES/MODULES

10.a – Learning Outcomes

By the end of this chapter, you will be able to:

- **Define** what a module is and explain its role in organizing and reusing Python code.
- **Differentiate** between built-in modules, external modules, and user-created modules.
- **Create** a simple Python module containing variables, functions, and classes.
- **Import** a module using the `import` keyword and access its contents using dot notation.
- **Import** specific items from a module using the `from ... import ...` syntax.
- **Use** the wildcard (*) import and explain why it is generally discouraged.
- **Alias** modules and imported items using the `as` keyword.
- **Remove** a module from the current namespace using the `del()` function.
- **Define** what a package is and explain the role of the `__init__.py` file.
- **Recognize** the difference between the standard library, PyPI packages, and local modules.

10.b – Introduction

As programs grow larger, placing all code in a single file becomes impractical. Python solves this problem through **modules** and **packages**, which allow you to organize code into reusable, maintainable units.

A **module** is a single Python file containing definitions and statements. A **package** is a directory containing multiple modules and a special `__init__.py` file. Together, they form the foundation of Python's code organization and reuse strategy.

This chapter covers:

- **Modules** – creating, importing, and using modules
- **Packages** – understanding how Python organizes modules into directories

Note: In this course, we constrain ourselves to modules available in the standard Python installation. We will avoid downloading and installing external packages.

10.c – Modules

A module is a file containing Python code that can define:

- variables
- functions
- classes

Modules allow you to organize and reuse/share code across different programs or projects.

10.c.1 – Examples of built-in modules

These are pre-installed during a standard python installation:

- `os`
- `abc`
- `random`
- `math`
- `json`
- `datetime`

10.c.2 – External modules

These need to be installed from pypi.org using `pip`:

- `matplotlib`
- `numpy`
- `pandas`
- `tensorflow`
- `flask`

10.c.3 – Creating your own Module

We will create a python code file that has a variable declaration and two function definitions. If the file is in the same folder that you are running the Python interpreter, then your usage does not need the path of the name, just use the name.

```
'''
This module serves to illustrate the purpose
and use of modules in the python language.
'''

pi = 3.141_592

def greet(name):
    return f'Hello {name}'
```

```
def calculate_area_of_triangle(base, height):  
    return base * height * 0.5
```

10.c.4 – Importing/Loading a Module

In order to access the code in a module, you will have to load that module. We will use the python `import` keyword with the name of the file to import.

```
>>> dir()  
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',  
 '__package__', '__spec__']  
  
>>> import module01  
>>> dir()  
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',  
 '__package__', '__spec__', 'module01']  
  
>>> dir(module01)  
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',  
 '__name__', '__package__', '__spec__', 'calculate_area_of_triangle',  
 'greet', 'pi']
```

Note: The `dir()` function without arguments lists names in the current namespace. With a module argument, it lists the names defined in that module.

10.c.4.1 – Importing specific items from a module

You can import a specific thing from a module. e.g. a function/class/constant You do this with the `from` keyword. It can save you from having to type the same thing many times.

```
>>> from module01 import pi, greet  
>>> pi  
3.141592  
  
>>> greet('Hao')  
'Hello Hao'  
  
>>> calculate_area_of_triangle(4, 7)  
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
NameError: name 'calculate_area_of_triangle' is not defined
```

Note: When you use `from module01 import pi, greet`, only `pi` and `greet` are brought into the current namespace. `calculate_area_of_triangle` is **not** available unless explicitly imported.

10.c.4.2 – Importing Everything

You can use the wildcard (*) to import everything from a module. Beware, this usually reduces code readability and also over-crowd the current workspace.

```
>>> from module01 import *
>>> pi
3.141592

>>> greet('Hao')
'Hello Hao'

>>> calculate_area_of_triangle(4, 7)
14.0
```

Best Practice: Avoid wildcard imports in production code. They make it unclear where names come from and can accidentally overwrite existing names.

10.c.4.3 – Aliasing

You can use the `as` keyword to give your module a different name. Then you will use this alias to access the module. This is useful if the name of the module changes and you have existing code that references the old module. You will use an alias also if it is tedious to type the full name of the module.

```
>>> from module01 import calculate_area_of_triangle as area
>>> area(4, 7)
14.0

>>> calculate_area_of_triangle(4, 7)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

```
NameError: name 'calculate_area_of_triangle' is not defined
```

Note: Once aliased, the original name is no longer available in the current namespace. You must use the alias.

10.c.5 – Using the Module

Statement to use the module will depend on how the module was imported.

```
>>> import module01
>>> module01.pi
3.141592
>>> module01.greet('Hao')
'Hello Hao'
```

Note: When you import the module itself (not specific items), you must prefix every access with the module name followed by a dot (.).

10.c.6 – Removing a Module

Use the `del()` function to remove a module from the current namespace.

```
del(module01)
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__']
```

Note: `del()` removes the **name binding** from the current namespace. It does **not** delete the module file from disk. The module can be re-imported later.

10.d – Packages

A **package** is a directory that contains multiple Python files (modules) and a special file named `__init__.py`. The `__init__.py` file indicates to Python that the directory should be treated as a package.

In this course we constrain ourselves to the modules that are available in the standard python installation, so we will avoid downloading and installing extra packages.

10.d.1 – Package Structure

```
my_package/  
    __init__.py  
    module_a.py  
    module_b.py  
    subpackage/  
        __init__.py  
        module_c.py
```

- `__init__.py` can be empty, or it can contain initialization code for the package.
- Modules within a package are imported using dot notation: `import my_package.module_a`

10.d.2 – Creating your own Package

This is not in the scope of this course. However, understanding the structure helps you navigate existing packages.

10.e – Summary

This chapter introduced modules and packages, the fundamental building blocks of code organization and reuse in Python.

Key takeaways:

- **Modules** are Python files (`.py`) containing variables, functions, and classes that can be imported and reused across programs.
- **Built-in modules** (`os`, `random`, `math`, `json`) come pre-installed with Python.
- **External modules** (`numpy`, `pandas`, `flask`) must be installed from PyPI using `pip`.
- `import module_name` loads the entire module; access items with `module_name.item`.
- `from module_name import item1, item2` loads specific items into the current namespace.
- `from module_name import *` loads everything — convenient but discouraged due to readability and namespace pollution.
- `import module_name as alias` or `from module_name import item as alias` creates a shorter or alternative name.
- `del(module_name)` removes the module name binding from the current namespace (does not delete the file).
- **Packages** are directories containing multiple modules and an `__init__.py` file, which tells Python to treat the directory as a package.

- **Packages enable hierarchical organization** of related modules, making large codebases manageable.
- **Best practice:** Prefer explicit imports (`import module` or `from module import specific_item`) over wildcard imports for clarity and maintainability.

10.f – Non coding exercises

1. What is a module?
2. What three things can a module define?
3. Why are modules useful in programming?
4. Name three built-in Python modules.
5. Name three external Python modules and where they are installed from.
6. How do you create your own module?
7. What keyword is used to import a module?
8. What is the difference between `import module01` and `from module01 import pi`?
9. What happens if you try to use a function that was not imported when using `from module01 import pi, greet`?
10. What does the wildcard (*) do in an import statement?
11. Why is wildcard importing discouraged?
12. What keyword is used to alias a module or imported item?
13. How do you access a variable from a module that was imported with `import module01`?
14. How do you access a variable from a module that was imported with `from module01 import pi`?
15. What does `del(module01)` do? Does it delete the file from disk?
16. What is a package?
17. What special file must a directory contain to be treated as a package?
18. What is the purpose of `__init__.py`?
19. How do you import a module from within a package?
20. What is the difference between the standard library and external packages?
21. Why does this course avoid external packages?
22. What does `dir()` return when called with no arguments?
23. What does `dir(module01)` return?
24. True or false: A module can contain classes.
25. True or false: A package can contain subpackages.
26. True or false: `del(module01)` permanently deletes the module from your computer.
27. True or false: `from module01 import *` is the recommended way to import modules.
28. True or false: An `__init__.py` file must contain code to be valid.
29. Why might you use an alias for a module?

30. What is the difference between a module and a package?